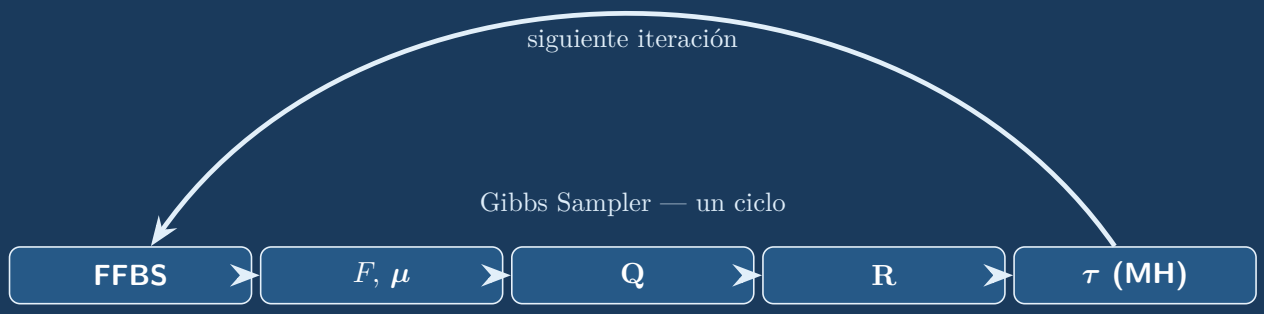
PROYECTO KJM — DOCUMENTACIÓN TÉCNICA

DNS / DNSS Bayesiano con Gibbs Sampler

Fundamentos teóricos, implementación en Python
y documentación de la función `fit_dns_bayes()`



Nelson-Siegel (DNS)	**Nelson-Siegel-Svensson (DNSS)**
$k = 3$ factores	$k = 4$ factores
τ_1 via MH 1D	(τ_1, τ_2) via MH 2D

Referencias principales: Carter & Kohn (1994) · Caldeira, Laurini & Portugal (2010) · Çakmaklı (2013)
Nelson & Siegel (1987) · Svensson (1994) · Diebold & Li (2006) · Gelman & Rubin (1992)

Índice

1. Introducción y motivación **3**
1.1. ¿Por qué un enfoque bayesiano? . . . 3
1.2. Estructura del notebook . . . 3

2. Modelo espacio-estado **4**
2.1. Formulación general . . . 4
2.2. Cargas Nelson-Siegel (DNS, $k = 3$) . . . 5
2.3. Cargas Nelson-Siegel-Svensson (DNSS, $k = 4$) . . . 5
2.4. Estructura de $\mathbf{F}$ . . . 5

3. Forward Filter Backward Sampler (FFBS) **5**
3.1. ¿Por qué FFBS y no muestreo univariado? . . . 6
3.2. Algoritmo completo . . . 6
3.3. Inicialización estacionaria . . . 7
3.4. Implementación en Python . . . 7

4. Distribuciones posteriores conjugadas **7**
4.1. Posterior de μ_i y f_i (Normal conjugada) . . . 7
4.2. Posterior de q_i (Inverse-Gamma conjugada) . . . 8
4.3. Posterior de r_n (Inverse-Gamma conjugada) . . . 8
4.4. Extensiones: Opciones A y C . . . 9
4.4.1. Opción A — $\mathbf{R}$ estructurada por grupos de plazos . . . 9
4.4.2. Opción C — Pesos en la posterior de $\mathbf{R}$ . . . 9

5. Metropolis-Hastings para τ **9**
5.1. ¿Por qué MH y no posterior conjugada? . . . 9
5.2. Log-verosimilitud marginal para MH . . . 9
5.3. Propuesta y ratio de aceptación . . . 10
5.4. Adaptación durante el burn-in . . . 10

6. Especificación de priors **10**

7. Diagnósticos de convergencia MCMC **11**
7.1. Estadístico $\hat{R}$ de Gelman-Rubin . . . 11
7.2. Effective Sample Size (ESS) . . . 12
7.3. Visualizaciones de diagnóstico . . . 12

8. Documentación de funciones **13**
8.1. Funciones de cargas . . . 13
8.2. Posteriors conjugadas . . . 13
8.3. Metropolis-Hastings para τ . . . 14
8.4. Especificación de priors . . . 14
8.5. Una cadena: `run_chain()` . . . 15
8.6. Multi-cadena: `run_chains_parallel()` . . . 15

8.7. Diagnósticos 16
8.8. Visualizaciones MCMC 16
8.9. Post-ajuste 16

9. Función principal `fit_dns_bayes()` 17
9.1. Firma completa 17
9.2. Descripción de parámetros 18
9.3. Estructura del retorno 18

10. Ejemplos de uso 19
10.1. Ejemplo 1 — DNS básico 19
10.2. Ejemplo 2 — DNSS con prior informativo y mejoras para plazos largos 20
10.3. Ejemplo 3 — Comparativa bayesiano vs KF 21
10.4. Ejemplo 4 — Uso programático sin gráficos 22

11. Referencia completa de funciones 23

12. Guía de decisiones 24

Referencias 24

1. Introducción y motivación

1.1. ¿Por qué un enfoque bayesiano?

El estimador de Máxima Verosimilitud (MLE) con Filtro de Kalman produce estimaciones *puntuales* de los factores $\boldsymbol{\beta}_{1:T}$ y los parámetros $\boldsymbol{\theta}$. Esto tiene dos consecuencias importantes en la práctica:

- Los **intervalos de confianza** de los factores no incorporan la incertidumbre sobre los parámetros $\boldsymbol{\theta}$ — son condicionales en $\hat{\boldsymbol{\theta}}$ y por tanto subestiman la incertidumbre total.
- El parámetro de forma τ se selecciona como un punto (via Grid Search), descartando la incertidumbre sobre su valor.

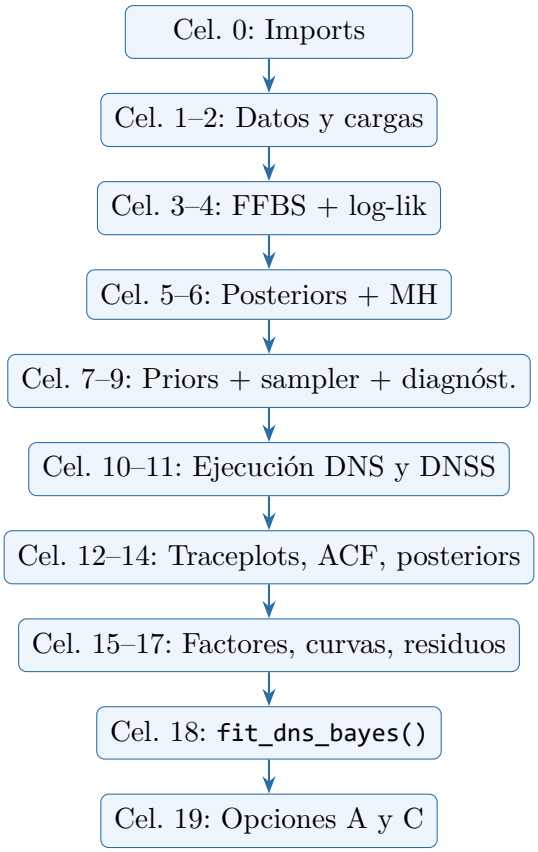
El enfoque bayesiano resuelve ambos problemas al **probar sobre toda la distribución posterior** $p(\boldsymbol{\beta}_{1:T}, \boldsymbol{\theta}, \boldsymbol{\tau} \mid \mathbf{Y})$ en lugar de optimizarla. La Tabla 1 resume las diferencias.

Cuadro 1: Comparación Kalman Filter + MLE vs. Gibbs Sampler bayesiano

Aspecto	**Kalman Filter + MLE**	**Gibbs Sampler (Bayes)**
Factores $\boldsymbol{\beta}_{1:T}$	Estimado puntual $\mathbb{E}\left[\boldsymbol{\beta}_t \mid \mathbf{Y}, \hat{\boldsymbol{\theta}}\right]$	Distribución completa $p(\boldsymbol{\beta}_{1:T} \mid \mathbf{Y})$
Parámetros $\boldsymbol{\theta}$	Optimización numérica (L-BFGS-B)	Distribución posterior $p(\boldsymbol{\theta} \mid \mathbf{Y})$
Incertidumbre	Solo la del estado ($P_{t\|t}$)	Paramétrica + de estado + de τ
Intervalos	Asintóticos, condicionales en $\hat{\boldsymbol{\theta}}$	Intervalos de credibilidad exactos
τ	Fijo via Grid Search	Distribución posterior via MH
Costo computacional	$O(T \cdot N^2 \cdot k^2)$	$O(n_{\text{iter}} \cdot T \cdot N^2 \cdot k^2)$
Información a priori	No incorporada	Incorporada mediante priors

1.2. Estructura del notebook

El notebook `DNS_DNSS_Bayesiano_Gibbs.ipynb` implementa el pipeline completo en 20 celdas organizadas en el orden de dependencia natural:



2. Modelo espacio-estado

2.1. Formulación general

El modelo trabaja en el marco de espacio-estado lineal gaussiano:

Modelo espacio-estado DNS/DNSS

Ecuación de observación:

$$\mathbf{y}_t = \mathbf{H}(\boldsymbol{\tau})\,\boldsymbol{\beta}_t + \boldsymbol{\varepsilon}_t, \qquad \boldsymbol{\varepsilon}_t \sim \mathcal{N}(\mathbf{0},\, \mathbf{R}) \tag{1}$$

Ecuación de transición:

$$\boldsymbol{\beta}_t = \boldsymbol{\mu} + \mathbf{F}(\boldsymbol{\beta}_{t-1} - \boldsymbol{\mu}) + \boldsymbol{\eta}_t, \qquad \boldsymbol{\eta}_t \sim \mathcal{N}(\mathbf{0},\, \mathbf{Q}) \tag{2}$$

donde $\mathbf{y}_t \in \mathbb{R}^N$ son los rendimientos observados en N plazos, $\boldsymbol{\beta}_t \in \mathbb{R}^k$ son los factores latentes ($k = 3$ para DNS, $k = 4$ para DNSS), $\mathbf{H}(\boldsymbol{\tau}) \in \mathbb{R}^{N\times k}$ son las cargas (variables en $\boldsymbol{\tau}$), $\boldsymbol{\mu} \in \mathbb{R}^k$ es la media de largo plazo, $\mathbf{F} \in \mathbb{R}^{k\times k}$ es la matriz de transición, $\mathbf{R} = \mathrm{diag}(r_1, \ldots, r_N)$ y $\mathbf{Q} = \mathrm{diag}(q_1, \ldots, q_k)$ son matrices diagonales.

2.2. Cargas Nelson-Siegel (DNS, $k = 3$)

La función de carga mapea cada plazo m_n a un vector de pesos para los tres factores:

$$\mathbf{H}_n(\tau_1) = \left[1, \quad \frac{1 - e^{-m_n/\tau_1}}{m_n/\tau_1}, \quad \frac{1 - e^{-m_n/\tau_1}}{m_n/\tau_1} - e^{-m_n/\tau_1}\right] \tag{3}$$

- $\beta_{0,t}$: **nivel** — carga constante = 1; domina en plazos largos.
- $\beta_{1,t}$: **pendiente** — carga decreciente de 1 a 0; spread largo-corto.
- $\beta_{2,t}$: **curvatura** — carga con joroba (*hump*) en $m^* \approx 1{,}793\,\tau_1$ años.

2.3. Cargas Nelson-Siegel-Svensson (DNSS, $k = 4$)

los autores extiende la especificación con un cuarto factor que captura una segunda curvatura, útil cuando existe una prima de plazo significativa:

$$\mathbf{H}_n(\tau_1, \tau_2) = \left[\mathbf{H}_n(\tau_1), \quad \frac{1 - e^{-m_n/\tau_2}}{m_n/\tau_2} - e^{-m_n/\tau_2}\right] \tag{4}$$

El segundo hump ocurre en $m^{**} \approx 1{,}793\,\tau_2$. Se impone la restricción $|\tau_1 - \tau_2| \geq 0{,}15$ para evitar colinealidad en las columnas de $\mathbf{H}$ que causaría mal condicionamiento.

2.4. Estructura de F

El notebook soporta tres estructuras para la matriz de transición, siguiendo la taxonomía de los autores:

Cuadro 2: Estructuras de **F** disponibles

Tipo	Params.	Muestreo	Referencia
`'diagonal'`	k	Normal conjugada truncada	Caldeira et al. (2010) P7
`'triangular'`	$k(k+1)/2$	Diagonal conjugada + MH off-diag	Extensión propia
`'completa'`	k^2	MH matricial	VAR(1) irrestricto

La especificación diagonal está respaldada empíricamente: los autores reportan una diferencia de RMSE $< 0{,}4$ pb entre la especificación diagonal (P7) y el VAR completo (P6), mientras que los autores la adopta explícitamente por su parsimonia.

3. Forward Filter Backward Sampler (FFBS)

3.1. ¿Por qué FFBS y no muestreo univariado?

En el Gibbs sampler para modelos espacio-estado, la opción más simple sería muestrear cada $\boldsymbol{\beta}_t$ condicionado en sus vecinos $\boldsymbol{\beta}_{t-1}$ y $\boldsymbol{\beta}_{t+1}$. Este enfoque produce cadenas con **autocorrelación extremadamente alta** porque los estados adyacentes están fuertemente correlacionados en la posterior.

El algoritmo FFBS de los autores muestrea **la trayectoria completa $\boldsymbol{\beta}_{1:T}$ en una sola pasada**, lo que equivale a muestrear directamente de la distribución condicional conjunta y elimina la autocorrelación interna entre estados.

3.2. Algoritmo completo

Algoritmo 3.1 (FFBS — Carter & Kohn (1994))**.** Dado $\boldsymbol{\theta} = (\boldsymbol{\mu}, \mathbf{F}, \mathbf{Q}, \mathbf{R}, \boldsymbol{\tau})$, muestrear $\boldsymbol{\beta}_{1:T} \sim p(\boldsymbol{\beta}_{1:T} \mid \mathbf{Y}, \boldsymbol{\theta})$.

Fase forward (Kalman Filter): Para $t = 1, \ldots, T$:

$$\hat{\boldsymbol{\beta}}_{t|t-1} = \boldsymbol{\mu} + \mathbf{F}(\hat{\boldsymbol{\beta}}_{t-1|t-1} - \boldsymbol{\mu}) \tag{5}$$
$$\mathbf{P}_{t|t-1} = \mathbf{F}\,\mathbf{P}_{t-1|t-1}\,\mathbf{F}^\top + \mathbf{Q} \tag{6}$$
$$\boldsymbol{v}_t = \mathbf{y}_t - \mathbf{H}\,\hat{\boldsymbol{\beta}}_{t|t-1} \tag{7}$$
$$\mathbf{S}_t = \mathbf{H}\,\mathbf{P}_{t|t-1}\,\mathbf{H}^\top + \mathbf{R} \tag{8}$$
$$\mathbf{K}_t = \mathbf{P}_{t|t-1}\,\mathbf{H}^\top\,\mathbf{S}_t^{-1} \tag{9}$$
$$\hat{\boldsymbol{\beta}}_{t|t} = \hat{\boldsymbol{\beta}}_{t|t-1} + \mathbf{K}_t\,\boldsymbol{v}_t \tag{10}$$
$$\mathbf{P}_{t|t} = (\mathbf{I} - \mathbf{K}_t\mathbf{H})\,\mathbf{P}_{t|t-1}\,(\mathbf{I} - \mathbf{K}_t\mathbf{H})^\top + \mathbf{K}_t\mathbf{R}\mathbf{K}_t^\top \tag{11}$$

Fase backward (muestreo): Muestrear $\boldsymbol{\beta}_T \sim \mathcal{N}(\hat{\boldsymbol{\beta}}_{T|T},\, \mathbf{P}_{T|T})$. Para $t = T-1, \ldots, 1$:

$$\mathbf{P}_{t+1|t} = \mathbf{F}\,\mathbf{P}_{t|t}\,\mathbf{F}^\top + \mathbf{Q} \tag{12}$$
$$\mathbf{G}_t = \mathbf{P}_{t|t}\,\mathbf{F}^\top\,\mathbf{P}_{t+1|t}^{-1} \tag{13}$$
$$\boldsymbol{\mu}_{t|T} = \hat{\boldsymbol{\beta}}_{t|t} + \mathbf{G}_t\Big(\boldsymbol{\beta}_{t+1} - \hat{\boldsymbol{\beta}}_{t+1|t}\Big) \tag{14}$$
$$\boldsymbol{\Sigma}_{t|T} = \mathbf{P}_{t|t} - \mathbf{G}_t\,\mathbf{P}_{t+1|t}\,\mathbf{G}_t^\top \tag{15}$$
$$\boldsymbol{\beta}_t \sim \mathcal{N}\big(\boldsymbol{\mu}_{t|T},\, \boldsymbol{\Sigma}_{t|T}\big) \tag{16}$$

Forma Joseph (ecuación 11)

La actualización de covarianza usa la forma Joseph $(I-KH)P(I-KH)^\top + KRK^\top$ en lugar de la forma estándar $(I-KH)P$. Aunque más costosa computacionalmente, garantiza que $\mathbf{P}_{t|t}$ permanezca simétrica definida positiva incluso con errores de redondeo acumulados, lo que es crítico en series largas.

3.3. Inicialización estacionaria

La condición inicial $\mathbf{P}_{0|0}$ se obtiene resolviendo la ecuación de Lyapunov discreta $\mathbf{P} = \mathbf{F}\mathbf{P}\mathbf{F}^\top + \mathbf{Q}$. Para $\mathbf{F}$ diagonal, la solución es explícita por factor:

$$P_{0,ii} = \frac{q_i}{1 - f_i^2}, \qquad i = 1, \ldots, k \tag{17}$$

3.4. Implementación en Python

La función central `_ffbs_diag()` está compilada con Numba JIT para $\mathbf{F}$ diagonal. El wrapper público `ffbs()` acepta cualquier k:

Celda 3 — ffbs() (extracto clave)

```python
def ffbs(mu, f_vec, q_vec, r_vec, H, yields):
    '''Wrapper publico de FFBS para F diagonal.
    Valido para k=3 (DNS) y k=4 (DNSS).'''
    return _ffbs_diag(
        np.asarray(mu,     dtype=np.float64),
        np.asarray(f_vec,  dtype=np.float64),  # diag(F)
        np.asarray(q_vec,  dtype=np.float64),  # diag(Q)
        np.asarray(r_vec,  dtype=np.float64),  # diag(R)
        np.asarray(H,      dtype=np.float64),  # (N, k)
        np.asarray(yields, dtype=np.float64),  # (T, N)
    )
```

Velocidad

Con Numba JIT activo, `_ffbs_diag()` corre aproximadamente $8\times$ más rápido que el equivalente Python puro. Para $T = 300$, $N = 8$, $k = 4$, el tiempo por iteración es $\approx 2\,\text{ms}$, lo que hace factibles 5,000 iteraciones en $\approx 10\,\text{s}$ por cadena.

4. Distribuciones posteriores conjugadas

La elegancia del Gibbs sampler para este modelo radica en que, *condicionado en la trayectoria* $\boldsymbol{\beta}_{1:T}$ (muestreada por FFBS), todos los demás parámetros tienen posteriors analíticas exactas.

4.1. Posterior de $\boldsymbol{\mu}$ y f_i (Normal conjugada)

El modelo de transición para el factor i es: $\beta_{i,t} = \mu_i + f_i(\beta_{i,t-1} - \mu_i) + \eta_{i,t}$, que puede reescribirse como:

$$\beta_{i,t} - f_i\beta_{i,t-1} = (1 - f_i)\mu_i + \eta_{i,t} \tag{18}$$

Posterior de μ_i dado f_i

Con prior $\mu_i \sim \mathcal{N}(m_{0i},\, V_{0i})$ y definiendo $\ell_{i,t} \equiv \beta_{i,t} - f_i\,\beta_{i,t-1}$:

$$\mu_i \mid f_i, \boldsymbol{\beta}, q_i \;\sim\; \mathcal{N}\big(\bar{m}_i,\; \bar{V}_i\big) \tag{19}$$

$$\bar{V}_i^{-1} = \frac{1}{V_{0i}} + \frac{(1-f_i)^2\,T_{\text{eff}}}{q_i}, \qquad \bar{m}_i = \bar{V}_i\left(\frac{m_{0i}}{V_{0i}} + \frac{1-f_i}{q_i}\sum_{t=1}^{T_{\text{eff}}} \ell_{i,t}\right) \tag{20}$$

donde $T_{\text{eff}} = T - 1$.

Posterior de f_i dado μ_i (Normal truncada)

Definiendo $a_{i,t} = \beta_{i,t} - \mu_i$:

$$f_i \mid \mu_i, \boldsymbol{\beta}, q_i \;\sim\; \mathcal{TN}_{(-1,1)}\big(\bar{f}_i,\; \bar{V}_{f_i}\big) \tag{21}$$

$$\bar{V}_{f_i}^{-1} = \frac{1}{V_{f_{0i}}} + \frac{\sum_t a_{i,t-1}^2}{q_i}, \qquad \bar{f}_i = \bar{V}_{f_i}\left(\frac{f_{0i}}{V_{f_{0i}}} + \frac{\sum_t a_{i,t}\,a_{i,t-1}}{q_i}\right) \tag{22}$$

La truncación en $(-1, 1)$ garantiza la estacionariedad del proceso AR(1) de cada factor. Se muestrea via la transformación de percentil de la Normal truncada (`scipy.stats.truncnorm`).

Observación 4.1. El muestreo se hace *secuencialmente* por factor ($i = 1, \ldots, k$) en el orden $\mu_i \to f_i$, actualizando el valor de μ_i antes de muestrear f_i. Esto es el *Gibbs dentro de Gibbs* y garantiza que los dos parámetros sean consistentes en cada iteración.

4.2. Posterior de q_i (Inverse-Gamma conjugada)

Los residuos de estado son $\eta_{i,t} = \beta_{i,t} - \mu_i - f_i(\beta_{i,t-1} - \mu_i)$.

Posterior de q_i

Con prior $q_i \sim \mathcal{IG}(a_0, b_0)$:

$$q_i \mid \boldsymbol{\beta}, \mu_i, f_i \;\sim\; \mathcal{IG}\left(a_0 + \frac{T_{\text{eff}}}{2},\;\; b_0 + \frac{1}{2}\sum_{t=1}^{T_{\text{eff}}} \eta_{i,t}^2\right) \tag{23}$$

4.3. Posterior de r_n (Inverse-Gamma conjugada)

Los residuos de medición son $\varepsilon_{n,t} = y_{t,n} - \mathbf{H}_n\,\boldsymbol{\beta}_t$.

Posterior de r_n

Con prior $r_n \sim \mathcal{IG}(a_0, b_0)$:

$$r_n \mid \boldsymbol{\beta}, \mathbf{Y} \;\sim\; \mathcal{IG}\left(a_0 + \frac{T}{2},\;\; b_0 + \frac{1}{2}\sum_{t=1}^{T} \varepsilon_{n,t}^2\right) \tag{24}$$

4.4. Extensiones: Opciones A y C

4.4.1. Opción A — R estructurada por grupos de plazos

En lugar de estimar N varianzas independientes, se reducen a 3 parámetros correspondiendo a grupos de plazos (corto/medio/largo). La posterior por grupo g recoge los residuos de todos los plazos $n \in \mathcal{G}_g$:

$$r_g \mid \boldsymbol{\beta}, \mathbf{Y} \sim \mathcal{IG}\left(a_0 + \frac{T\,|\mathcal{G}_g|}{2},\ \ b_0 + \frac{1}{2}\sum_{n\in\mathcal{G}_g}\sum_{t=1}^{T}\varepsilon_{n,t}^2\right) \tag{25}$$

Esto impone una estructura parsimoniosa en $\mathbf{R}$ que refleja el hecho conocido de que la liquidez y los errores de medición varían sistemáticamente por tramo de curva.

4.4.2. Opción C — Pesos en la posterior de R

Asignar peso $w_n > 1$ a un plazo n equivale a reducir el hiperparámetro b_0 efectivo, lo que hace que la posterior concentre más masa en valores pequeños de r_n (menor varianza de medición $\rightarrow$ mejor ajuste):

$$r_n^{(w)} \mid \boldsymbol{\beta}, \mathbf{Y} \sim \mathcal{IG}\left(a_0 + \frac{T}{2},\ \ \frac{b_0}{w_n} + \frac{1}{2}\sum_{t=1}^{T}\varepsilon_{n,t}^2\right) \tag{26}$$

5. Metropolis-Hastings para $\boldsymbol{\tau}$

5.1. ¿Por qué MH y no posterior conjugada?

El parámetro $\boldsymbol{\tau}$ entra de forma *no lineal* en la matriz de cargas $\mathbf{H}(\boldsymbol{\tau})$ — aparece en exponenciales y cocientes — por lo que no existe familia conjugada. Se usa el algoritmo Metropolis-Hastings, que solo requiere evaluar la función objetivo (la log-verosimilitud marginal) en la propuesta.

5.2. Log-verosimilitud marginal para MH

Al proponer un nuevo $\boldsymbol{\tau}^*$, es incorrecto evaluar la log-verosimilitud con la trayectoria $\boldsymbol{\beta}_{1:T}$ que fue muestreada bajo el $\boldsymbol{\tau}$ anterior: esa trayectoria es inconsistente con el nuevo valor propuesto. La solución es usar la **log-verosimilitud marginal** $p(\mathbf{Y} \mid \boldsymbol{\tau}^*, \boldsymbol{\theta})$ donde $\boldsymbol{\beta}_{1:T}$ está integrado vía el Filtro de Kalman:

$$\log p(\mathbf{Y} \mid \boldsymbol{\tau}, \boldsymbol{\theta}) = -\frac{TN}{2}\log(2\pi) - \frac{1}{2}\sum_{t=1}^{T}\left[\log|\mathbf{S}_t| + \boldsymbol{v}_t^\top \mathbf{S}_t^{-1}\boldsymbol{v}_t\right] \tag{27}$$

donde $\boldsymbol{v}_t$ y $\mathbf{S}_t$ son la innovación y su varianza del KF con $\mathbf{H}(\boldsymbol{\tau})$. El $\log|\mathbf{S}_t|$ se calcula via descomposición de Cholesky: $\log|\mathbf{S}_t| = 2\sum_n \log L_{t,nn}$, lo que es numéricamente estable incluso cuando $\mathbf{S}_t$ está mal condicionada.

5.3. Propuesta y ratio de aceptación

MH para τ — Propuesta random walk en log-escala

Para garantizar positividad sin truncar el espacio:

$$\text{DNS:} \quad \log\tau_1^* = \log\tau_1 + \sigma_1\, z, \quad z \sim \mathcal{N}(0,1) \tag{28}$$

$$\text{DNSS:} \quad \begin{pmatrix}\log\tau_1^* \\ \log\tau_2^*\end{pmatrix} = \begin{pmatrix}\log\tau_1 \\ \log\tau_2\end{pmatrix} + \begin{pmatrix}\sigma_1 z_1 \\ \sigma_2 z_2\end{pmatrix}, \quad z_1, z_2 \overset{iid}{\sim} \mathcal{N}(0,1) \tag{29}$$

Ratio de aceptación ($\alpha = \text{mín}(1, e^{\log\alpha})$):

$$\log\alpha = \underbrace{[\log p(\mathbf{Y} \mid \boldsymbol{\tau}^*, \boldsymbol{\theta}) - \log p(\mathbf{Y} \mid \boldsymbol{\tau}, \boldsymbol{\theta})]}_{\text{ratio log-lik}} + \underbrace{[\log p(\boldsymbol{\tau}^*) - \log p(\boldsymbol{\tau})]}_{\text{ratio prior}} \tag{30}$$

El Jacobiano del cambio $\log\tau \to \tau$ cancela porque la propuesta es simétrica en log-escala.

Observación 5.1. La propuesta independiente en log-escala para $(log\tau_1^*, \log\tau_2^*)$ es equivalente a un MH 2D con propuesta diagonal. La tasa óptima de aceptación es $\approx 44\,\%$ para $d = 1$ y $\approx 23\,\%$ para $d = 2$ según los autores.

5.4. Adaptación durante el burn-in

Durante el burn-in, σ se ajusta cada $n_{\text{burnin}}/5$ iteraciones:

$$\sigma \leftarrow \begin{cases} 0{,}70\,\sigma & \text{si } \hat{r} < r_{\text{target}} - 0{,}12 \\ 1{,}35\,\sigma & \text{si } \hat{r} > r_{\text{target}} + 0{,}15 \\ \sigma & \text{en otro caso} \end{cases} \tag{31}$$

con $r_{\text{target}} = 0{,}40$ (DNS) y $r_{\text{target}} = 0{,}28$ (DNSS), y σ acotado en $[0{,}005, 0{,}60]$.

6. Especificación de priors

Los priors siguen la filosofía *débilmente informativos* de los autores y los autores: centrados en valores económicamente plausibles pero lo suficientemente difusos para que los datos dominen.

Cuadro 3: Especificación de priors — función `make_priors()`

Parámetro	Prior	Justificación
f_i	$\mathcal{TN}_{(-1,1)}(0{,}90,\ 0{,}10^2)$	Alta persistencia típica en DNS; truncación garantiza estacionariedad
μ_i	$\mathcal{N}(m_{0i},\ 25)$	Difuso; centrado en valores históricos plausibles ($\mu_0 \approx 5\,\%$, $\mu_1 \approx -2\,\%$)
q_i, r_n	$\mathcal{IG}(0{,}01,\ 0{,}01)$	Casi no-informativo (Jeffreys-like); permite que los datos dominen la escala
τ_1 (uniforme)	$\mathcal{U}(0{,}15,\ 5{,}0)$	Bounds económicos; hump en $[0{,}27, 9{,}0]$ años
τ_1 (normal)	$\mathcal{N}(1{,}78,\ 0{,}40^2)$	Informativo; hump esperado en $\approx 3{,}2$ años (típico Tesoros EEUU)
τ_2 (normal)	$\mathcal{N}(0{,}60,\ 0{,}08^2)$	Fuertemente informativo; evita colinealidad con τ_1 en plazos cortos

Celda 7 — make_priors() con prior normal informativos

```
# Prior uniforme (difuso)
priors = make_priors('DNSS', prior_tau_type='uniforme')

# Prior normal informativos para tau (cuando se conoce el regimen)
priors = make_priors('DNSS', prior_tau_type='normal',
                     tau1_mu=1.78, tau1_sd=0.40,   # hump beta2 en ~3.2 Y
                     tau2_mu=0.60, tau2_sd=0.08)   # hump beta3 en ~1.1 Y
```

7. Diagnósticos de convergencia MCMC

7.1. Estadístico $\hat{R}$ de Gelman-Rubin

El $\hat{R}$ compara la varianza *dentro* de cadenas (W) con la varianza *entre* cadenas (B). Si las cadenas han convergido a la misma distribución, $\hat{R} \approx 1$.

$\hat{R}$ de Gelman-Rubin

Con m cadenas de longitud n cada una, para un parámetro escalar ψ:

$$B = \frac{n}{m-1}\sum_{j=1}^{m}(\bar{\psi}_j - \bar{\psi})^2 \quad \text{(varianza entre cadenas)} \tag{32}$$

$$W = \frac{1}{m}\sum_{j=1}^{m} s_j^2 \quad \text{(varianza dentro de cadenas)} \tag{33}$$

$$\widehat{\mathrm{Var}[\psi \mid Y]} = \frac{n-1}{n}W + \frac{B}{n} \tag{34}$$

$$\hat{R} = \sqrt{\frac{\widehat{\mathrm{Var}[\psi \mid Y]}}{W}} \tag{35}$$

Criterios de convergencia:

- $\hat{R} < 1{,}05$: convergencia excelente (`OK`)
- $\hat{R} < 1{,}10$: convergencia aceptable (`~`)
- $\hat{R} \geq 1{,}10$: no convergió — extender iteraciones (`XX`)

7.2. Effective Sample Size (ESS)

La autocorrelación en las muestras MCMC reduce el número efectivo de muestras independientes. El ESS corrige por esta autocorrelación:

$$\mathrm{ESS} = \frac{n}{1 + 2\sum_{k=1}^{\infty}\hat{\rho}_k} \tag{36}$$

donde $\hat{\rho}_k$ es la autocorrelación estimada en lag k. La suma se trunca cuando $|\hat{\rho}_k| < 0{,}05$.

Cuadro 4: Interpretación del ESS

ESS	Interpretación
> 400	Buena precisión de Monte Carlo
100–400	Aceptable; considerar más iteraciones
< 100	Mezcla pobre; aumentar `n_iter` o aplicar thinning

7.3. Visualizaciones de diagnóstico

El notebook genera tres bloques de diagnóstico MCMC:

1. **Traceplots** (`plot_traceplots`): evolución de los parámetros por iteración. Una cadena bien mezclada debe explorar el espacio, solaparse entre cadenas y no tener tendencia sistemática.
2. **ACF de la cadena** (`plot_acf_mcmc`): función de autocorrelación de las muestras. Un decaimiento rápido a cero indica buena mezcla. Se reporta el ESS por parámetro.

3. **Distribuciones posteriores** (`plot_posteriors`): histogramas con KDE, media posterior, e intervalo de credibilidad al 95 %.

8. Documentación de funciones

8.1. Funciones de cargas

Celda 2 — Cargas NS y NSS

```python
def ns_loadings(maturities, tau1):
    '''Cargas Nelson-Siegel. Shape (N, 3). [nivel, pendiente, curvatura]'''
    m  = np.asarray(maturities, dtype=float)
    mt = m / tau1; e = np.exp(-mt); c = (1.0 - e) / mt
    return np.column_stack([np.ones_like(m), c, c - e])

def nss_loadings(maturities, tau1, tau2):
    '''Cargas Nelson-Siegel-Svensson. Shape (N, 4).
    Condicion: |tau1 - tau2| >= 0.15 para evitar colinealidad.'''
    m   = np.asarray(maturities, dtype=float)
    mt1 = m/tau1; mt2 = m/tau2
    e1  = np.exp(-mt1); e2 = np.exp(-mt2)
    c1  = (1.0 - e1) / mt1
    return np.column_stack([np.ones_like(m), c1, c1-e1, (1.0-e2)/mt2-e2])

def get_H(maturities, tau1, tau2, model):
    '''Wrapper unificado. model: "DNS" o "DNSS".'''
    if model == 'DNS':
        return ns_loadings(maturities, tau1)
    return nss_loadings(maturities, tau1, tau2)
```

8.2. Posteriors conjugadas

Celda 5 — Muestreo de F, μ, Q, R

```python
# F diagonal y mu — Normal conjugada
f_new, mu_new = sample_F_mu_diagonal(
    betas,                          # (T, k) trayectoria FFBS actual
    f_vec, q_vec,                   # parametros actuales
    prior_f_mean, prior_f_var,      # prior de f_i
    prior_mu_mean, prior_mu_var)    # prior de mu_i

# Q — Inverse-Gamma conjugada (por factor)
```

```
q_new = sample_Q(betas, mu, f_vec, prior_Q_a, prior_Q_b)

# R — libre: N varianzas independientes
r_new = sample_R(yields, betas, H, prior_R_a, prior_R_b,
                 obs_weights=None)   # obs_weights: Opcion C

# R — grupos: 3 varianzas por tramo de curva (Opcion A)
r_new = sample_R_groups(yields, betas, H, prior_R_a, prior_R_b,
                        maturities,
                        group_thresholds=(3.0, 10.0))
```

8.3. Metropolis-Hastings para τ

Celda 6 — MH para τ

```
# DNS: MH 1D para tau1
tau1, loglik_new, accepted = sample_tau_MH_dns(
    tau1, yields, mu, f_vec, q_vec, r_vec,
    maturities,
    prop_std=0.08,            # sigma de la propuesta en log-escala
    tau_bounds=(0.15, 5.0),   # limites del prior uniforme
    prior_tau=('uniforme', 0.15, 5.0))

# DNSS: MH 2D para (tau1, tau2)
tau1, tau2, loglik_new, accepted = sample_tau_MH_dnss(
    tau1, tau2, yields, mu, f_vec, q_vec, r_vec,
    maturities,
    prop_std=np.array([0.08, 0.06]),
    tau_bounds=(0.15, 5.0),
    prior_tau=('normal', 1.78, 0.40, 0.60, 0.08),
    sep_min=0.15)             # separacion minima |tau1 - tau2|
```

8.4. Especificación de priors

Celda 7 — make_priors()

```
def make_priors(
    model,                        # 'DNS' o 'DNSS'
    prior_tau_type = 'uniforme',  # 'uniforme' o 'normal'
    tau1_mu=1.78, tau1_sd=0.40,   # prior normal para tau1
    tau2_mu=0.60, tau2_sd=0.08,   # prior normal para tau2 (DNSS)
    tau_lo=0.15,  tau_hi=5.0,     # bounds del prior uniforme
)
# Retorna dict con:
# 'prior_f_mean'  : (k,)   media prior de f_i
# 'prior_f_var'   : (k,)   varianza prior de f_i
```

```
# 'prior_mu_mean' : (k,)   media prior de mu_i
# 'prior_mu_var'  : (k,)   varianza prior de mu_i
# 'prior_Q_a/b'   : float  hiperparametros IG para Q
# 'prior_R_a/b'   : float  hiperparametros IG para R
# 'prior_tau'     : tuple  especificacion del prior de tau
# 'tau_bounds'    : tuple  (tau_lo, tau_hi)
```

8.5. Una cadena: `run_chain()`

Celda 8 — run_chain() (5 bloques del Gibbs)

```
def run_chain(
    yields, maturities, priors,
    n_iter, n_burnin, seed,
    tau1_init=None,          # tau inicial (None -> OLS two-step)
    tau2_init=None,
    prop_std_init=None,      # sigma inicial del MH (None -> auto)
    adapt_mh=True,           # adaptar sigma durante burn-in
    obs_weights=None,        # (N,) Opcion C
    r_mode='libre',          # 'libre' o 'grupos' (Opcion A)
    group_thresholds=(3.0, 10.0),
    verbose=False,
):
# Retorna dict con:
# 'mu'    : (n_store, k)    muestras de mu
# 'f_vec' : (n_store, k)    muestras de diag(F)
# 'q_vec' : (n_store, k)    muestras de diag(Q)
# 'r_vec' : (n_store, N)    muestras de diag(R)
# 'tau1'  : (n_store,)      muestras de tau1
# 'tau2'  : (n_store,)      muestras de tau2 (solo DNSS)
# 'betas' : (n_store, T, k) trayectorias de factores
# 'mh_final_rate' : float   tasa de aceptacion final MH
# 'prop_std_final': array   sigma final adaptado
```

8.6. Multi-cadena: `run_chains_parallel()`

Celda 9 — run_chains_parallel()

```
chains = run_chains_parallel(
    yields, maturities, priors,
    n_iter=3000, n_burnin=1000,
    n_chains=4,              # >= 2 para calcular R-hat
    n_jobs=-1,               # -1 = todos los nucleos disponibles
    obs_weights=None,        # Opcion C
    r_mode='libre',          # Opcion A
    group_thresholds=(3.0, 10.0),
```

```
    verbose=True,
)
# Retorna: lista de n_chains dicts (resultado de run_chain)
```

8.7. Diagnósticos

Celda 9 — Diagnósticos de convergencia

```
# Tabla completa R-hat y ESS
df_diag = compute_diagnostics(chains)
print_diagnostics(chains)

# R-hat para un parametro especifico
rhat_tau1 = gelman_rubin(chains, 'tau1')
rhat_f0   = gelman_rubin(chains, 'f_vec', idx=0)

# ESS para una cadena
ess = effective_sample_size(chains[0]['tau1'])
```

8.8. Visualizaciones MCMC

Celdas 11--14 — Gráficos de diagnóstico

```
# Traceplots de todas las cadenas
plot_traceplots(chains, titulo='(DNS)')

# ACF de la cadena con ESS por parametro
plot_acf_mcmc(chains, n_chain=0, max_lag=60)

# Distribuciones posteriores con IC 95%
plot_posteriors(chains, priors)

# Posterior de tau y hump location
plot_posterior_tau(chains, maturities)
```

8.9. Post-ajuste

Celdas 15--16 — Factores y residuos

```
# Estadisticos de la distribucion posterior de beta_t
beta_mean, beta_q025, beta_q975, beta_std = extract_posterior_betas(
    chains)
# beta_mean  : (T, k)  media posterior  E[beta_t | Y]
# beta_q025  : (T, k)  percentil 2.5%
```

```
# beta_q975  : (T, k)  percentil 97.5%

# Grafico de factores con bandas de credibilidad
plot_factores_bayes(chains, dates)

# Calcular ajuste y residuos con media posterior
fit = compute_posterior_fit(chains, yields, maturities)
print(f"RMSE = {fit['rmse_total']*100:.3f} pb")

# Analisis de residuos (mismo formato que KF notebook)
analisis_residuos_b(fit)
plot_heatmap_residuos_b(fit)
```

9. Función principal `fit_dns_bayes()`

9.1. Firma completa

Celda 18 — Firma de fit_dns_bayes()

```
def fit_dns_bayes(
    dates,                          # DatetimeIndex o array de fechas
    yields,                         # (T, N) DataFrame o ndarray
    maturities,                     # (N,) plazos en anos
    # ── Modelo ──────────────────────────────────────────
    model             = 'DNS',      # 'DNS' (k=3) o 'DNSS' (k=4)
    # ── MCMC ────────────────────────────────────────────
    n_iter            = 3000,       # total de iteraciones (incl. burn-in)
    n_burnin          = 1000,       # iteraciones a descartar
    n_chains          = 4,          # >= 2 para calcular R-hat
    # ── Priors de tau ───────────────────────────────────
    prior_tau_type    = 'uniforme', # 'uniforme' o 'normal'
    tau1_mu           = 1.78,       # media prior normal tau1
    tau1_sd           = 0.40,       # std prior normal tau1
    tau2_mu           = 0.60,       # media prior normal tau2 (DNSS)
    tau2_sd           = 0.08,       # std prior normal tau2 (DNSS)
    tau_lo            = 0.15,       # bound inferior prior uniforme
    tau_hi            = 5.0,        # bound superior prior uniforme
    # ── MH ──────────────────────────────────────────────
    prop_std_init     = None,       # sigma inicial propuesta MH (None=auto)
    # ── Mejoras para plazos largos ──────────────────────
    obs_weights       = None,       # (N,) Opcion C: pesos en posterior de R
```

```
    r_mode            = 'libre',     # 'libre' o 'grupos' (Opcion A)
    group_thresholds = (3.0, 10.0),# fronteras de grupos en anos
    # ── Control ──────────────────────────────────────────────
    plot              = True,        # generar todos los graficos
    verbose           = True,        # imprimir progreso
    n_jobs            = -1,          # nucleos para joblib
)
```

9.2. Descripción de parámetros

Parámetro	Default	Descripción
`model`	`'DNS'`	`'DNS'` para Nelson-Siegel ($k = 3$) o `'DNSS'` para Svensson ($k = 4$).
`n_iter`	3000	Total de iteraciones MCMC incluyendo burn-in. Para análisis de producción se recomiendan 5,000–10,000.
`n_burnin`	1000	Iteraciones descartadas. Típicamente $\sim 30\,\%$ del total.
`n_chains`	4	Número de cadenas paralelas. Mínimo 2 para calcular $\hat{R}$; 4 es el estándar en la literatura.
`prior_tau_type`	`'uniforme'`	`'uniforme'`: prior plano en `(tau_lo, tau_hi)`. `'normal'`: prior normal informativo, útil cuando se conoce el régimen de la curva.
`obs_weights`	`None`	**Opción C.** Array $(N,)$ de pesos por plazo para la posterior de R. `None` = todos iguales. Plazos con $w_n > 1$ reciben menor varianza de medición $\rightarrow$ mejor ajuste.
`r_mode`	`'libre'`	**Opción A.** `'libre'`: N varianzas independientes. `'grupos'`: 3 varianzas por grupos de plazos (corto/medio/largo).
`group_thresholds`	`(3.0, 10.0)`	Fronteras de grupos en años para `r_mode='grupos'`. Default: corto $\leq$ 3Y, medio $\leq$ 10Y, largo $>$ 10Y.
`plot`	`True`	Si `True`, genera automáticamente los 8 bloques de gráficos (traceplots, ACF, posteriors, τ, factores, curvas, heatmap, residuos).

9.3. Estructura del retorno

La función retorna un diccionario con tres grupos de claves:

Estructura del dict de retorno

```
res = fit_dns_bayes(...)

# ── Acceso a muestras MCMC ──────────────────────────────
res['chains']       # lista de n_chains dicts con muestras brutas
res['priors']       # dict de priors usados

# Concatenar muestras de todas las cadenas:
tau1_samp = np.concatenate([ch['tau1'] for ch in res['chains']])
f0_samp   = np.concatenate([ch['f_vec'][:,0] for ch in res['chains']])

# ── Diagnosticos de convergencia ────────────────────────
res['diagnostics']  # pd.DataFrame con R-hat y ESS por parametro

# ── Ajuste in-sample (media posterior) ──────────────────
res['fit']['beta_mean']    # (T, k)  factores latentes E[beta_t|Y]
res['fit']['beta_q025']    # (T, k)  percentil 2.5% de beta_t
res['fit']['beta_q975']    # (T, k)  percentil 97.5% de beta_t
res['fit']['tau1_pm']      # float   media posterior de tau1
res['fit']['tau2_pm']      # float   media posterior de tau2 (DNSS)
res['fit']['fitted']       # (T, N)  rendimientos ajustados
res['fit']['residuals']    # (T, N)  residuos de medicion
res['fit']['rmse_total']   # float   RMSE total (mismas unidades que
    yields)
res['fit']['rmse_mat']     # (N,)    RMSE por plazo
```

10. Ejemplos de uso

10.1. Ejemplo 1 — DNS básico

Uso mínimo — DNS con priors por defecto

```
import numpy as np
import pandas as pd

# Cargar datos (ajustar ruta)
df = pd.read_csv("rendimientos.csv", parse_dates=["Date"])
df = df.set_index("Date")
yield_cols  = ["Y1", "Y2", "Y3", "Y5", "Y10", "Y20"]
maturities  = np.array([1, 2, 3, 5, 10, 20], dtype=float)

# Ajuste DNS bayesiano
res = fit_dns_bayes(
    dates      = df.index,
```

```python
    yields     = df[yield_cols].values,
    maturities = maturities,
    model      = 'DNS',
    n_iter     = 3000,
    n_burnin   = 1000,
    n_chains   = 4,
)
print(f"RMSE = {res['fit']['rmse_total']*100:.3f} pb")

# Extraer factores
beta_t = res['fit']['beta_mean']     # (T, 3)
nivel       = beta_t[:, 0]           # nivel (tasa de largo plazo)
pendiente   = beta_t[:, 1]           # pendiente (spread)
curvatura   = beta_t[:, 2]           # curvatura (joroba)

# Incertidumbre sobre tau1
tau1_samp = np.concatenate([ch['tau1'] for ch in res['chains']])
print(f"tau1: media={tau1_samp.mean():.3f}  "
      f"IC95%=[{np.percentile(tau1_samp,2.5):.3f}, "
      f"{np.percentile(tau1_samp,97.5):.3f}]")
```

10.2. Ejemplo 2 — DNSS con prior informativo y mejoras para plazos largos

DNSS con Opciones A y C + prior informativo

```python
# Pesos extra a plazos >= 10 anos (Opcion C)
mats    = np.array([1, 2, 3, 5, 10, 20], dtype=float)
pesos   = np.where(mats >= 10, 2.0, 1.0)

res = fit_dns_bayes(
    dates       = df.index,
    yields      = df[yield_cols].values,
    maturities  = mats,
    model       = 'DNSS',
    n_iter      = 5000,
    n_burnin    = 1500,
    n_chains    = 4,
    # Prior informativo (para curvas tipicas Tesoros EEUU):
    prior_tau_type = 'normal',
    tau1_mu=1.78, tau1_sd=0.40,    # hump beta2 en ~3.2Y
    tau2_mu=0.60, tau2_sd=0.08,    # hump beta3 en ~1.1Y
    # Opciones para plazos largos:
    obs_weights      = pesos,          # Opcion C
    r_mode           = 'grupos',       # Opcion A
    group_thresholds = (3.0, 10.0),
    plot = True,
)
```

10.3. Ejemplo 3 — Comparativa bayesiano vs KF

Comparativa usando los resultados de ambos notebooks

```python
# Suponer que ya se tienen res_kf (del KF notebook)
# y res_bayes (de este notebook)
print("── Comparativa RMSE por plazo ──────────────────────────────")
print(f"  {'Plazo':>5}  {'KF':>8}  {'Bayes':>8}  {'Delta':>8}")
for i, m in enumerate(maturities):
    r_kf    = res_kf['rmse_mat'][i]               * 100
    r_bayes = res_bayes['fit']['rmse_mat'][i]      * 100
    print(f"  {m:>4.0f}Y  {r_kf:>8.3f}  {r_bayes:>8.3f}  "
          f"{r_bayes-r_kf:>+8.3f}")

# Comparar incertidumbre de factores:
# KF: IC basado en P_{t|T}[i,i]
# Bayes: percentiles de muestras MCMC
beta_mean, q025, q975, _ = extract_posterior_betas(res_bayes['chains'])
bs_kf  = res_kf['beta_smoothed']
Ps_kf  = res_kf['P_smoothed']

# Ancho medio del intervalo de beta_0:
ancho_bayes = (q975[:,0] - q025[:,0]).mean()
ancho_kf    = (2 * 1.96 * np.sqrt(Ps_kf[:,0,0])).mean()
```

```python
print(f"\nAncho IC beta0:  KF={ancho_kf:.4f}   Bayes={ancho_bayes:.4f}"
    )
print("(Bayes es mas amplio: incorpora incertidumbre de theta y tau)")
```

10.4. Ejemplo 4 — Uso programático sin gráficos

Modo silencioso para loops o backtesting

```python
# Ajuste rapido sin graficos (para comparar multiples especificaciones)
resultados = {}
for modelo in ['DNS', 'DNSS']:
    for tipo_prior in ['uniforme', 'normal']:
        llave = f"{modelo}_{tipo_prior}"
        res = fit_dns_bayes(
            dates=df.index, yields=df[yield_cols].values,
            maturities=mats,
            model=modelo,
            n_iter=2000, n_burnin=500, n_chains=2,
            prior_tau_type=tipo_prior,
            plot=False, verbose=False,
        )
        resultados[llave] = {
            'rmse'  : res['fit']['rmse_total'] * 100,
            'rhat_max': res['diagnostics']['Rhat'].max(),
            'ess_min' : res['diagnostics']['ESS_min'].min(),
        }

for k, v in resultados.items():
    print(f"  {k:<18}: RMSE={v['rmse']:.3f}pb  "
          f"Rhat_max={v['rhat_max']:.3f}  ESS_min={v['ess_min']:.0f}")
```

11. Referencia completa de funciones

Cuadro 6: Todas las funciones del notebook en orden de dependencia

Función	Retorno	Descripción
`ns_loadings(m, $\tau_1$)`	$(N, 3)$	Cargas Nelson-Siegel
`nss_loadings(m, $\tau_1$, $\tau_2$)`	$(N, 4)$	Cargas Svensson
`get_H(m, $\tau_1$, $\tau_2$, model)`	(N, k)	Wrapper unificado
`ols_init_tau(yields, m, model)`	float, float	τ inicial via OLS two-step
`_ffbs_diag(...)`	(T, k)	FFBS Numba JIT (F diagonal)
`ffbs(...)`	(T, k)	Wrapper público de FFBS
`_kf_loglik_diag(...)`	float	Log-lik marginal KF (JIT)
`kf_loglik(...)`	float	Wrapper público log-lik
`sample_F_mu_diagonal(..`	$(k,)$, $(k,)$	Posterior conjugada F, μ
`sample_Q(...)`	$(k,)$	Posterior IG para Q
`sample_R(...)`	$(N,)$	Posterior IG para R (libre)
`sample_R_groups(...)`	$(N,)$	Posterior IG para R (grupos)
`_log_prior_tau(...)`	float	Log-prior de τ
`sample_tau_MH_dns(...)`	float, float, bool	MH 1D para τ_1
`sample_tau_MH_dnss(...)`	float, float, float, bool	MH 2D para (τ_1, τ_2)
`make_priors(...)`	dict	Construye especificación de priors
`initialize_chain(...)`	dict	Inicialización OLS two-step
`run_chain(...)`	dict	Una cadena Gibbs completa
`run_chains_parallel(...`	list[dict]	Multi-cadena con joblib
`gelman_rubin(chains, key)`	float	Estadístico $\hat{R}$
`effective_sample_size(s`	float	ESS de una serie
`compute_diagnostics(cha`	DataFrame	Tabla $\hat{R}$ y ESS
`print_diagnostics(chair`	—	Imprime tabla de diagnósticos
`plot_traceplots(chains)`	—	Traceplots por parámetro
`plot_acf_mcmc(chains)`	—	ACF con ESS por parámetro
`plot_posteriors(chains)`	—	Histogramas posteriors
`plot_posterior_tau(chai`	—	Posterior τ y hump
`extract_posterior_betas`	$(T, k) \times 4$	Media, Q2.5 %, Q97.5 %, std
`plot_factores_bayes(cha`	—	Factores con IC bayesiano
`compute_posterior_fit(c`	dict	Ajuste con media posterior
`analisis_residuos_b(fit`	—	Análisis completo de residuos
`plot_heatmap_residuos_b`	—	Heatmap residuos en el tiempo
`fit_dns_bayes(...)`	dict	**Función principal**

12. Guía de decisiones

Cuadro 7: ¿Cuándo usar cada opción?

Situación	Configuración recomendada	Justificación
Análisis exploratorio	`n_iter=1000, n_chains=2`	Rápido, solo verificar convergencia
Producción / publicación	`n_iter=5000+, n_chains=4`	$\hat{R} < 1{,}05$, ESS > 400
Curva con forma conocida	`prior_tau='normal'`	Reduce exploración innecesaria de MH
Curva con prima de plazo	`model='DNSS'`	Segundo factor de curvatura
Residuos grandes en largo	`obs_weights + r_mode='grupos'`	Opciones A y C combinadas
Datos de alta frecuencia	`r_mode='grupos'`	Parsimonia, menos parámetros en R
Comparar con KF	`plot=True` + `plot_comparativa_bayes_kf()`	Visualiza ganancia en incertidumbre

Referencias

- **Nelson, C.R. & Siegel, A.F.** (1987). Parsimonious Modeling of Yield Curves. *Journal of Business*, 60(4), 473–489.
- **Svensson, L.E.O.** (1994). Estimating and Interpreting Forward Interest Rates: Sweden 1992–1994. *NBER Working Paper* No. 4871.
- **Diebold, F.X. & Li, C.** (2006). Forecasting the Term Structure of Government Bond Yields. *Journal of Econometrics*, 130(2), 337–364.
- **Carter, C.K. & Kohn, R.** (1994). On Gibbs Sampling for State Space Models. *Biometrika*, 81(3), 541–553.
- **Caldeira, J.F., Laurini, M.P. & Portugal, M.S.** (2010). Bayesian Inference Applied to Dynamic Nelson-Siegel Model. *Brazilian Review of Econometrics*, 30(1).
- **Çakmaklı, C.** (2013). Modelling the Term Structure with a Bayesian Dynamic Nelson-Siegel Model. Erasmus University Rotterdam Working Paper.
- **Gelman, A. & Rubin, D.B.** (1992). Inference from Iterative Simulation Using Multiple Sequences. *Statistical Science*, 7(4), 457–511.
- **Roberts, G.O. & Rosenthal, J.S.** (2001). Optimal Scaling for Various Metropolis-Hastings Algorithms. *Statistical Science*, 16(4), 351–367.
- **Hamilton, J.D.** (1994). *Time Series Analysis*. Princeton University Press. Capí-

tulo 13: State-Space Models and the Kalman Filter.

- **BIS** (2005). Zero-Coupon Yield Curves: Technical Documentation. *BIS Papers* No. 25, Bank for International Settlements.